


# Bases de Dados

## ÍNDICES

©José Barateiro 2024
Adaptado de Silberschatz, Korth e Sudarschan

# Conceitos básicos

- Mecanismos de indexação usados para acelerar o acesso aos dados.
  - Por exemplo, catálogo de autores numa biblioteca
- **Chave de pesquisa (search key)** - conjunto de atributos usados para procurar registos num ficheiro
- Um **ficheiro de índices (index file)** consiste em registos (chamados **entradas do índice - index entries**) na forma




- Os ficheiros de índices são, normalmente, muito menores do que os ficheiros de dados
- Dois tipos básicos de índices:
  - **Índices ordenados (ordered indices):** as chaves de pesquisa são armazenadas por ordem
  - **Índices de hash (hash indices):** as chaves de pesquisa são distribuídas uniformemente entre "buckets" usando uma "função de hash"

# Métricas de avaliação de índices

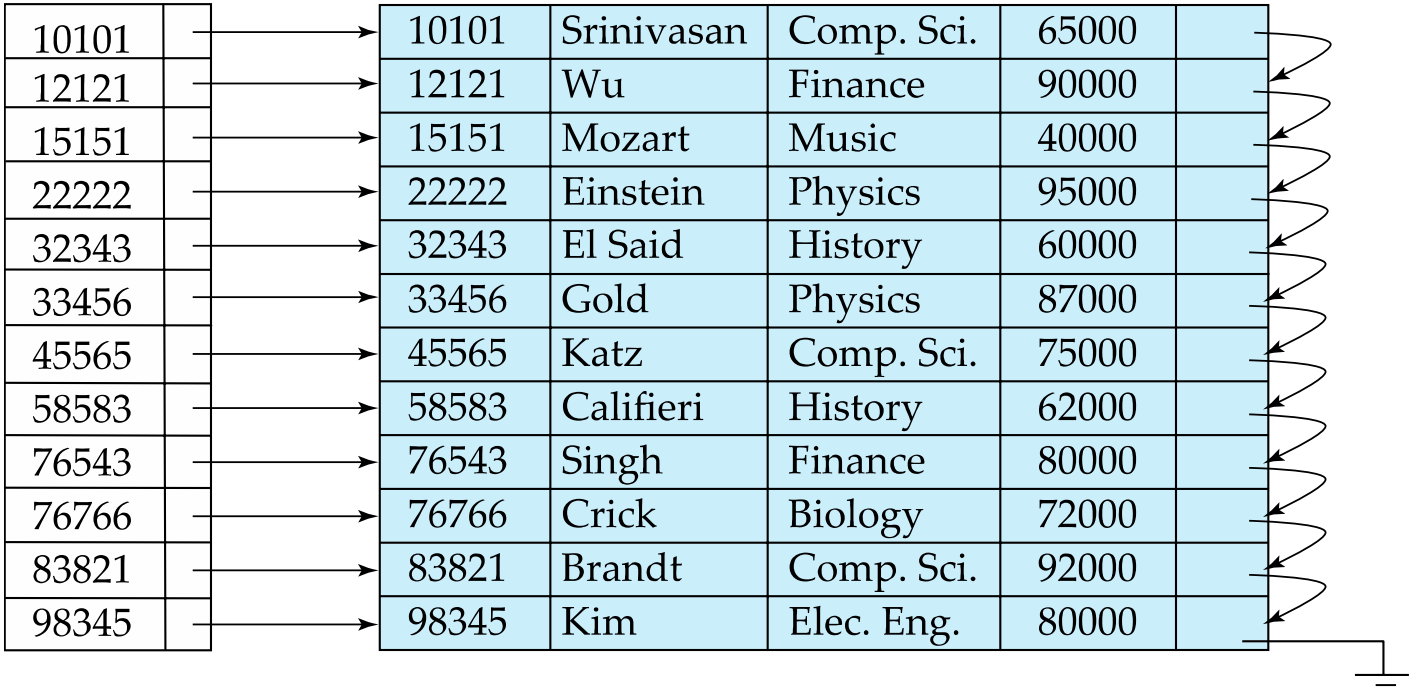
- Tipos de acesso suportados de forma eficiente. Por exemplo,
  - Registos com um valor específico num atributo
  - Registos com um valor no atributo que se enquadra num intervalo de valores
- Tempo de acesso
- Tempo de inserção
- Tempo de eliminação
- Espaço usado (*overhead*)

# Índices ordenados

- Num **índice ordenado,** as entradas do índice são armazenadas ordenadas pelo valor da chave de pesquisa.
- **Clustering index :** num ficheiro ordenado sequencialmente, o clustering index é o índice cuja *search key* determina a ordem sequencial do ficheiro.
  - Também chamado de **índice primário (primary index)**
  - A *searh key* de um índice primário geralmente é, mas não necessariamente, a chave primária.
- **Índice secundário (secondary index)**: um índice cuja *search key* especifica uma ordem diferente da ordem sequencial do ficheiro. Também chamado de **non clustering index.**
- **Index-sequential file:** ficheiro sequencial ordenado numa search key, com um clustering index nessa seach key.

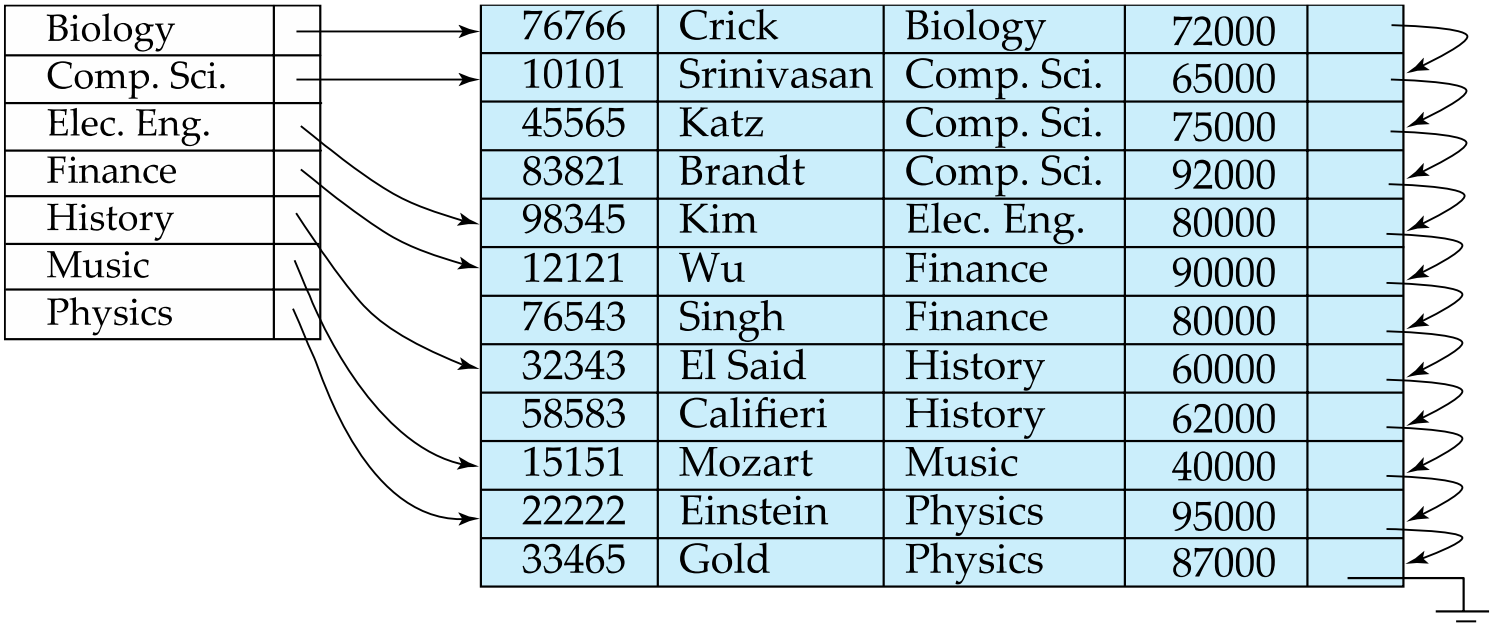
# Index files densos

- **Dense index** — Existe uma entrada de índice (index entry) para cada valor da search key
- Por exemplo, índice no atributo *ID* da relação *instrutor*



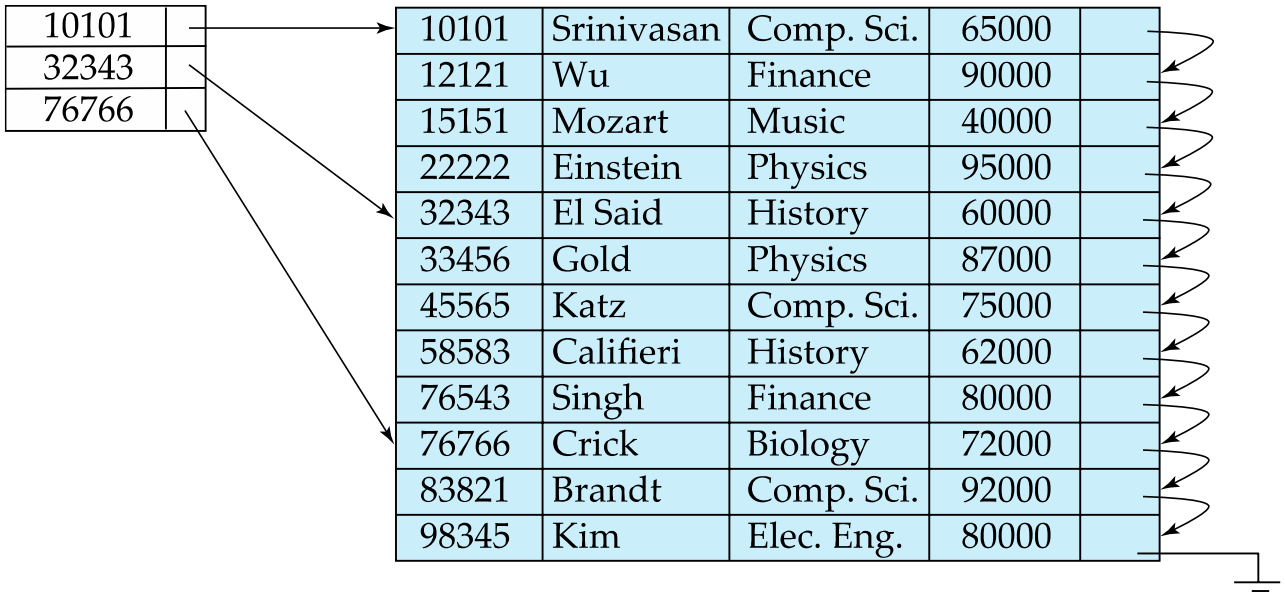
# Index files densos (cont.)

Índice denso em *dept_name*, com ficheiro de dados *instrutor* ordenado em *dept_name*



# Index files esparsos

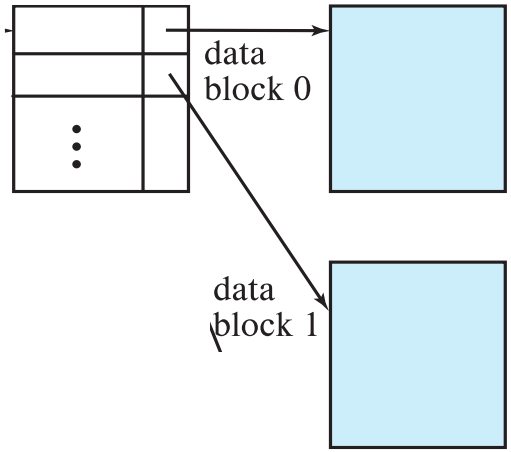
- **Índice esparso (sparse index)**: contém entradas de índice apenas para alguns valores da *searh key*.
  - Aplicável quando os registos são ordenados sequencialmente na search key
- Para localizar um registo com valor de search key *K*:
  - Encontre a entrada de índice com o maior valor de search key <= *K*
  - Percorrer o ficheiro, sequencialmente, começando no registo para o qual a entrada de índice aponta



# Index files esparsos (cont.)

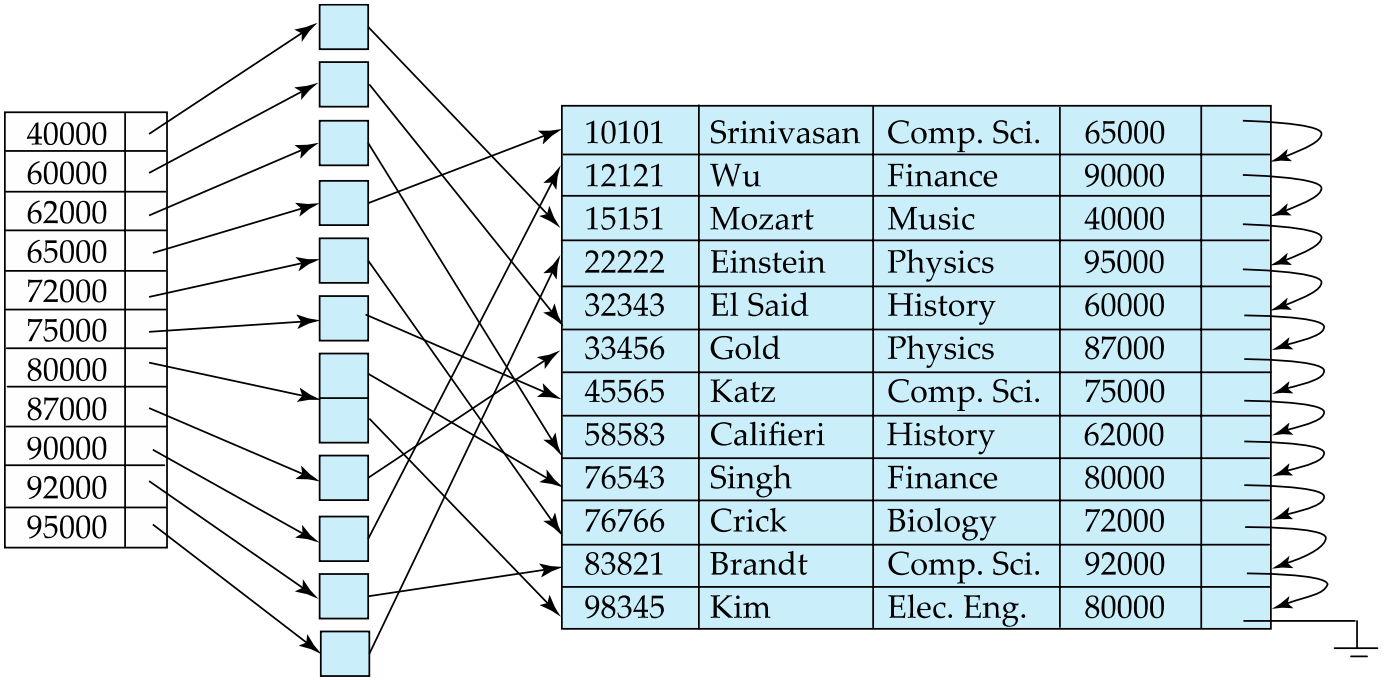
- Comparado com índices densos:
  - Menos espaço e menos custos de manutenção para inserções e eliminações.
  - Geralmente mais lento que o índice denso para localizar registos.

- **Boa troca** :
  - para clustered index: índice esparso com uma entrada de índice para cada bloco no ficheiro, correspondendo ao menor valor de search key no bloco.



# Exemplo de índices secundários (non-clustering)

- Índice secundário no campo salário do instrutor

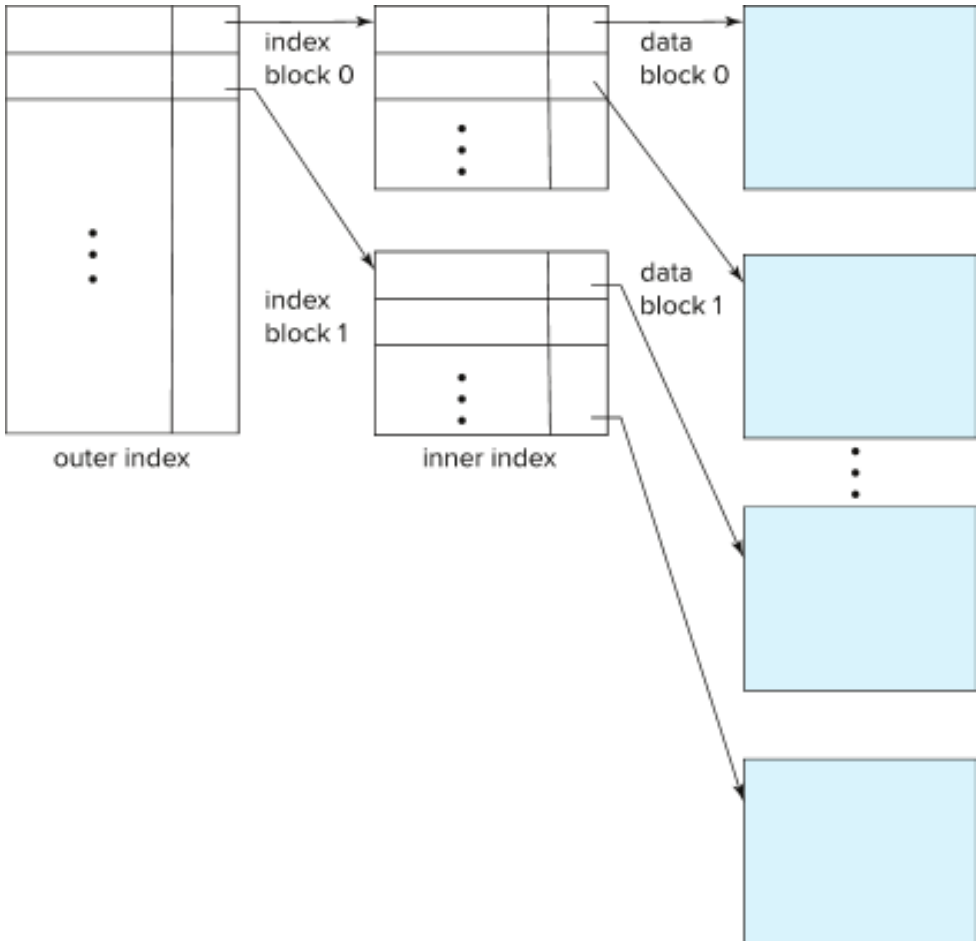



- Entradas do índice apontam para para um bucket (conjunto) que contém ponteiros para todos os registos reais com aquele valor de *search key*.
- Os índices secundários devem ser densos

# Índice multinível (multilevel index)

- Se o índice não estiver em memória, o acesso torna-se menos eficiente
- Solução: trate o índice mantido em disco como um ficheiro sequencial e construa um índice esparso para esse ficheiro.
  - índice externo (outer index) – um índice esparso sobre o índice básico
  - índice interno (inner index) – o ficheiro de índice básico
- Se até mesmo o índice externo for grande demais para caber na memória principal, outro nível de índice poderá ser criado, e assim por diante
- Índices em todos os níveis devem ser mantidos e atualizados

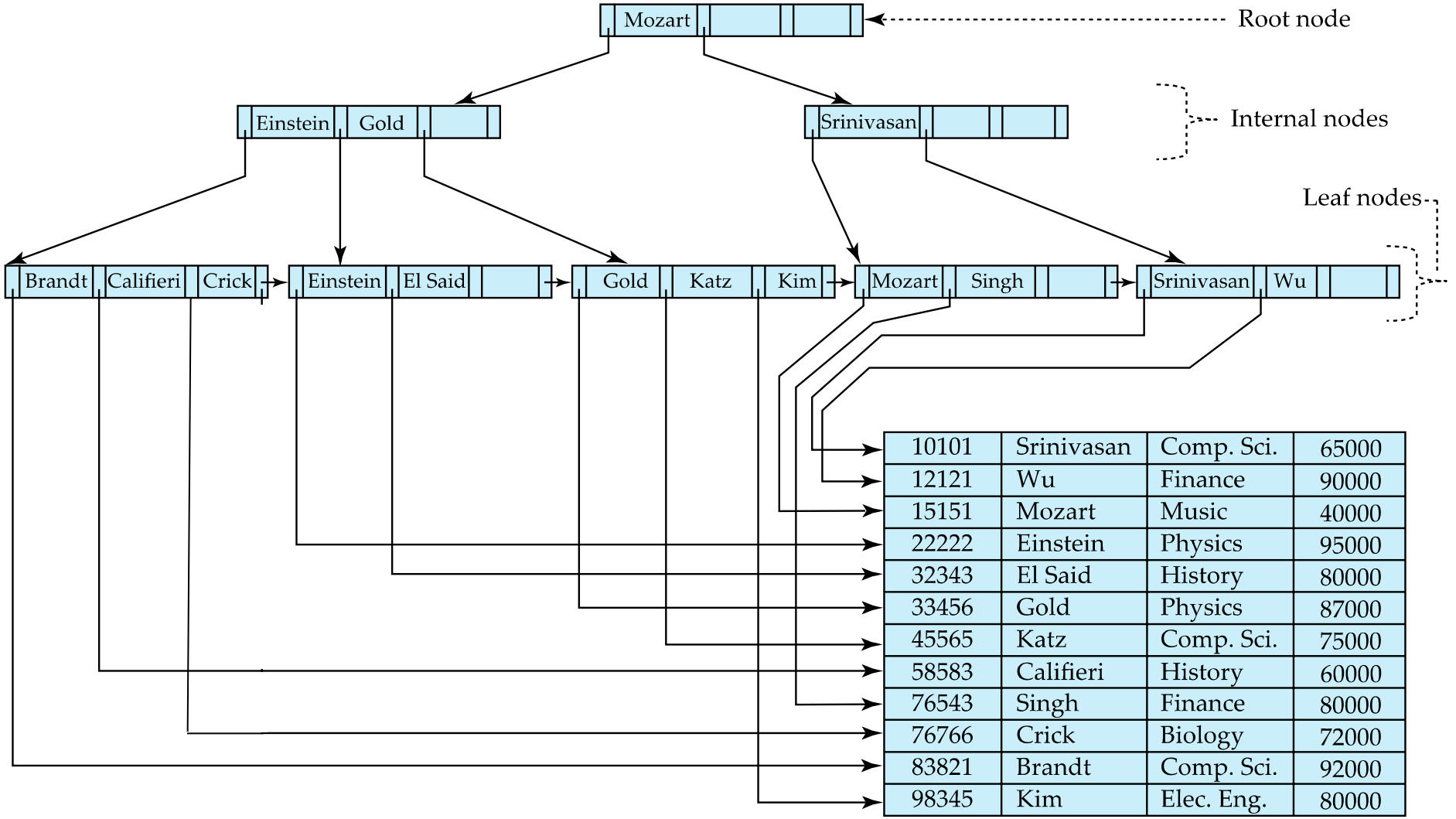
# Índice multinível (cont.)



# Índices com chaves compostas (multiple keys)

- **Chave de pesquisa composta (composite search key)**
  - Por exemplo, índice na relação *instrutor* em atributos (*name, ID*)
  - Os valores são ordenados lexicograficamente
    - Por exemplo (João, 12121) < (João, 13514) e (João, 13514) < (Pedro, 11223)
  - Pode consultar apenas *o name* ou ( *name, ID* )

# Exemplo de B $^+$-Tree




| | | | |
|---|---|---|---|
| 10101 | Srinivasan | Comp. Sci. | 65000 |
| 12121 | Wu | Finance | 90000 |
| 15151 | Mozart | Music | 40000 |
| 22222 | Einstein | Physics | 95000 |
| 32343 | El Said | History | 80000 |
| 33456 | Gold | Physics | 87000 |
| 45565 | Katz | Comp. Sci. | 75000 |
| 58583 | Califieri | History | 60000 |
| 76543 | Singh | Finance | 80000 |
| 76766 | Crick | Biology | 72000 |
| 83821 | Brandt | Comp. Sci. | 92000 |
| 98345 | Kim | Elec. Eng. | 80000 |

# $B^+$ Tree (continuação)

A $B^+$- tree é uma árvore que satisfaz as seguintes propriedades:

- Todos os caminhos da raiz até às folhas têm o mesmo comprimento
- Cada nó que não é uma raiz ou uma folha tem entre $\lceil n/2 \rceil$ e $n$ filhos.
- Um nó folha tem entre $\lceil (n-1)/2 \rceil$ e $n-1$ valores
- Casos especiais:
  - Se a raiz não for uma folha, ela tem pelo menos 2 filhos.
  - Se a raiz for uma folha (ou seja, não houver outros nós na árvore), ela pode ter entre 0 e $(n-1)$ valores.

# Estrutura do nó B $^+$-Tree

- Nó típico




  - $K_i$ são os valores da *search key*
  - $P_i$ são ponteiros para filhos (para nós não-folhas) ou ponteiros para registos ou grupos de registos (para nós-folhas).

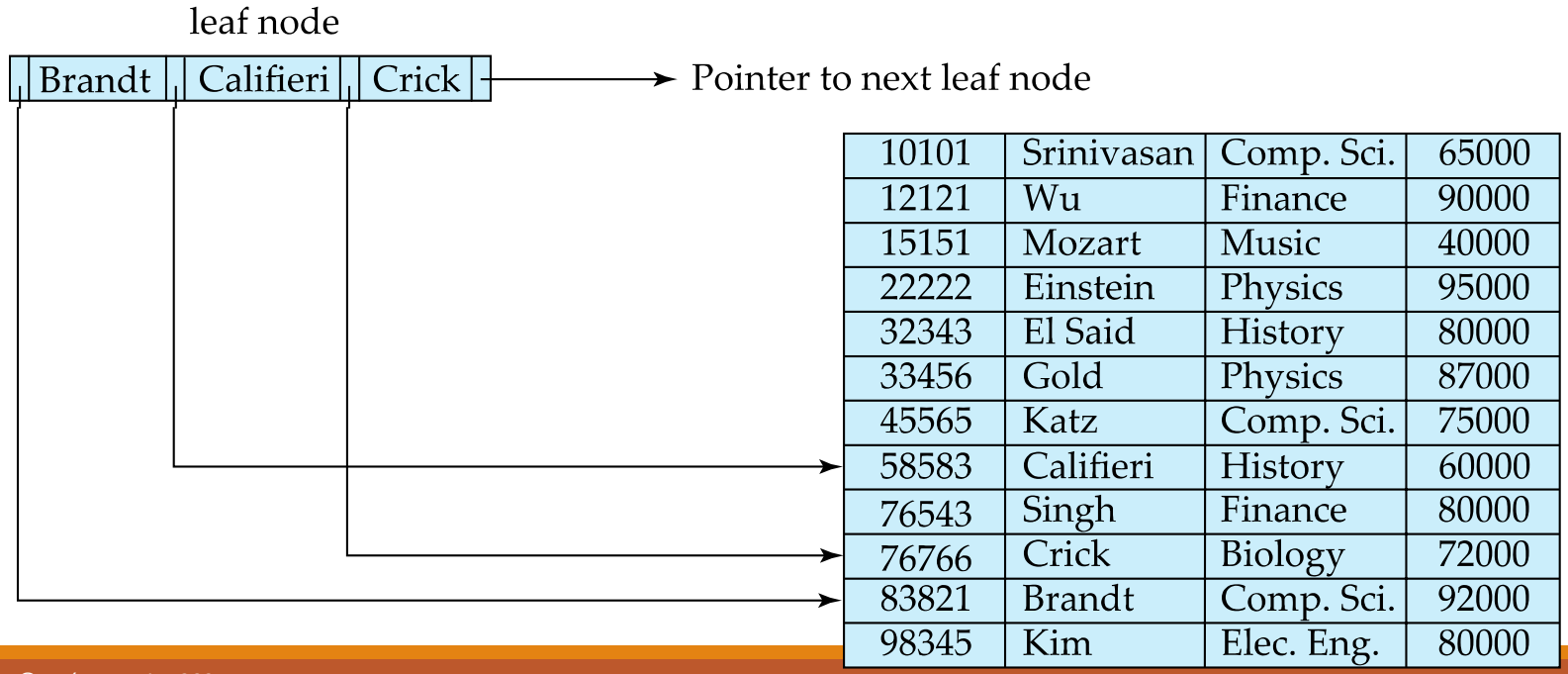
- As chaves de pesquisa num nó são ordenadas

$$K_1 < K_2 < K_3 < \ldots < K_{n-1}$$

# Nós de folhas em árvores B+ - Tree

Propriedades de um nó folha:

- Para i = 1, 2, . . ., n– 1, o ponteiro $P_i$ aponta para um registo de ficheiro com valor de search key $K_i$ ,
- Se $L_i$ , $L_j$ são nós folha e i < j, os valores das search keys de $L_i$ são menores ou iguais aos valores das search keys de $L_j$
- $P_n$ aponta para o próximo nó folha na ordem da search key




| | | | |
|---|---|---|---|
| 10101 | Srinivasan | Comp. Sci. | 65000 |
| 12121 | Wu | Finance | 90000 |
| 15151 | Mozart | Music | 40000 |
| 22222 | Einstein | Physics | 95000 |
| 32343 | El Said | History | 80000 |
| 33456 | Gold | Physics | 87000 |
| 45565 | Katz | Comp. Sci. | 75000 |
| 58583 | Califieri | History | 60000 |
| 76543 | Singh | Finance | 80000 |
| 76766 | Crick | Biology | 72000 |
| 83821 | Brandt | Comp. Sci. | 92000 |
| 98345 | Kim | Elec. Eng. | 80000 |

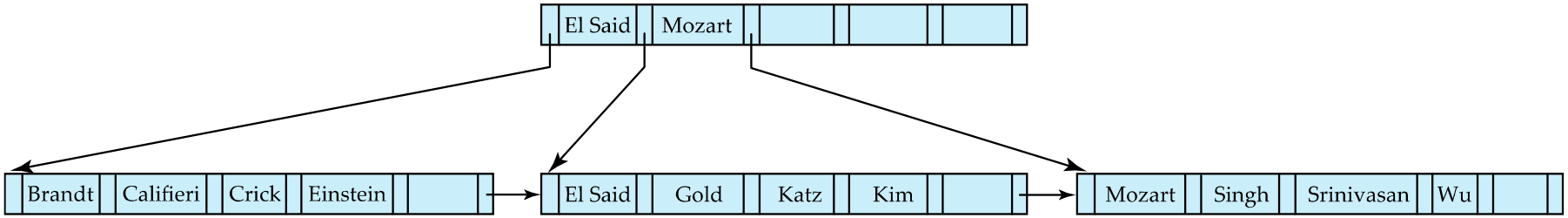
# Nós não-folhas em árvores B $^+$ - Tree

- Os nós não-folha formam um índice esparso multinível nos nós-folha. Para um nó não-folha com *m* ponteiros:
  - Todas as search keys na subárvore para a qual $P_1$ aponta são menores que $K_1$
  - Para $2 \leq i \leq n - 1$, todas as chaves de pesquisa na subárvore para a qual $P_i$ aponta têm valores maiores ou iguais a $K_{i-1}$ e menores que $K_i$
  - Todas as chaves de pesquisa na subárvore para a qual $P_n$ aponta têm valores maiores ou iguais a $K_{n-1}$
  - Estrutura geral



# Exemplo de árvore B $^+$ - Tree

B $^+$-tree para *instrutor* ( $n$ = 6)




- Nós folha devem ter entre 3 e 5 valores
  ($\lceil (n-1)/2 \rceil$ e $n-1$, com $n$ = 6).
- Nós não-folha, além da raiz, devem ter entre 3 e 6 filhos
  ($\lceil n/2 \rceil$ e $n$ com $n$ =6).
- A raiz deve ter pelo menos 2 filhos.

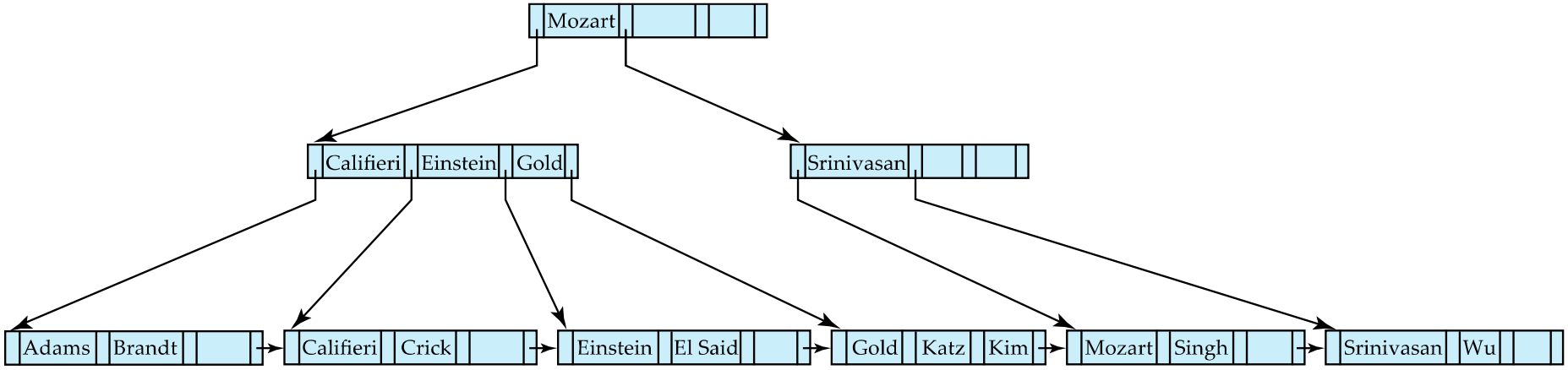
# Observações sobre B $^{+}$ - Trees

- Os níveis não-folha da árvore formam uma hierarquia de índices esparsos.
- A árvore contém um número relativamente pequeno de níveis
  - O nível abaixo da raiz tem pelo menos $2 * \lceil n/2 \rceil$ valores
  - O próximo nível tem pelo menos $2 * \lceil n/2 \rceil * \lceil n/2 \rceil$ valores
  - ...etc.
  - Se houver *K* valores de search keys, a profundidade da árvore não será maior que $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$
  - assim as pesquisas são realizadas de forma eficiente.
- Inserções e eliminações podem ser tratadas com eficiência, pois o índice pode ser reestruturado em tempo logarítmico.

# Queries em B $^{+}$- Trees

**function** *find*(*v*)

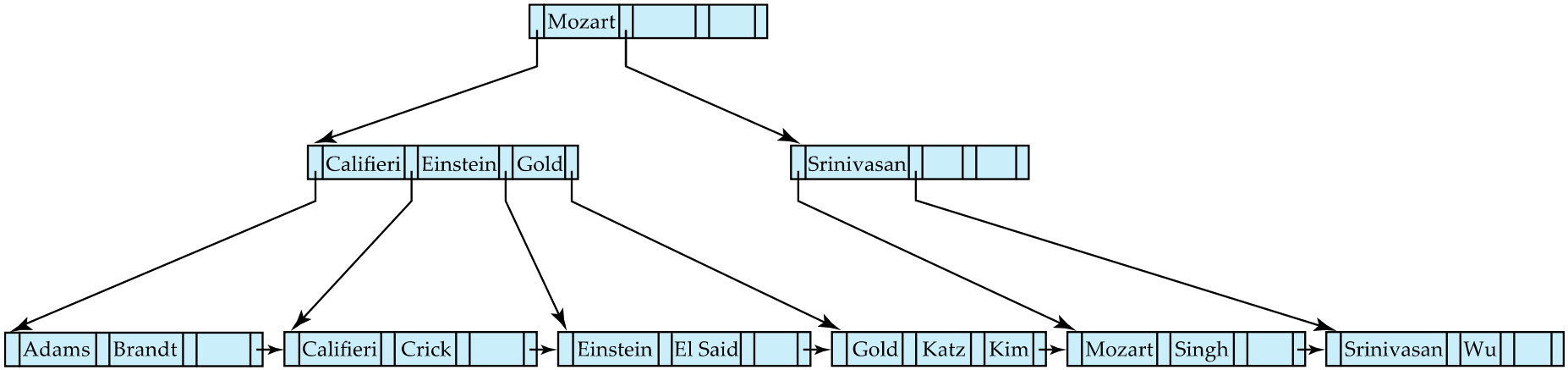
1. *C=root*
2. **while** (C is not a leaf node)
    1. Let *i* be least number s.t. $V \leq C.K_i$.
    2. **if** there is no such number *i then*
    3. Set *C* = *last non-null pointer in C*
    4. **else if** ($v$ = C.$K_i$ ) Set C = $P_{i+1}$
    5. **else set** *C* = C.$P_i$
3. **if** for some *i*, $K_i = V$ **then** return C.$P_i$
4. **else** return null /* no record with search-key value *v* exists. */



# Queries em B +- Trees (Cont.)

**Consultas de intervalos** encontram todos os registos com valores de *search key* num determinado intervalo

- *findRange* (*lb, ub*) que retorna o conjunto de todos esses registos
- Começa por percorrer o índice para encontra lb, tal como o find(v) do slide anterior (a folha pode não conter o valor lb, mas contém os valores imediatamente anteriores a lb que estão no índice )
- Seguidamente, itera nas folhas seguinte, obtendo todos os apontadores até ultrapassar o valor ub.



# Queries em B $^+$- Trees (cont.)

- Se houver $K$ valores de search keys, a profundidade da árvore não pode ser maior do que $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$.
- Normalmente, um nó tem o mesmo tamanho de um bloco de disco, tipicamente 4 kilobytes
  - e $n$ normalmente é em torno de 100 (40 bytes por entrada de índice).
- Com 1 milhão de valores de search keys e $n$ = 100
  - no máximo $log_{50}(1.000.000) = 4$ nós são acedidos para percorrer da raiz até a folha.

# Search keys não exclusivas

- Se uma *search key* $a_i$ não é única, crie um índice numa chave composta ($a_i$ , $A_p$ ), que é única
  - $A_p$ pode ser uma chave primária, ID de registo ou qualquer outro atributo que garanta exclusividade
- A pesquisa por $a_i = v$ pode ser implementada por uma pesquisa de intervalo na chave composta, com intervalo ( $v$, $-\infty$ ) a ( $v$, $+\infty$ )
- Mas mais operações de I/O são necessárias para encontrar os registos
  - Se o índice for *clustering*, todos os acessos serão sequenciais
  - Se o índice for *non-clustering*, cada acesso pode precisar de uma operação de I/O